

Chris Stevens

Professor Neil Kalinowski

CS-499

19 July 2026

Milestone 2 Narrative

The project selected for this enhancement was from CS-340 and was created in June 2025. The original application included a Python CRUD module for a MongoDB collection and a Dash application for providing analytics to a user. This was a good project to work on because it already provided value to the customer, but it was not built as a full application platform. This gave me an opportunity to port the project from Python/Dash into a MEVN stack and add new functionality with a frontend interface for creating, updating, and deleting animal records. As part of this transfer, I also included deployment using Docker, user authentication, and secure communication.

All these changes were successfully implemented, although the authentication is still partially staged until the database updates are completed for a user collection. A single account is currently hardcoded, but the backend work for express-session is implemented and allows logging in. The user collection and password storage will be completed as part of the database enhancement.

The following course outcomes are supported by these changes:

- Demonstrate an ability to use well-founded and innovative techniques, skills, and tools in computing practices for the purpose of implementing computer solutions that deliver value and accomplish industry-specific goals (software engineering/design/database)

Full-stack development tools and deployment practices were used to improve the original application. Porting the project to a MEVN stack, creating API-based CRUD operations, integrating the existing Dash dashboard concept, and using Docker Compose all support a more complete and maintainable application. The original project was useful, but the enhanced version is closer to an application that could be maintained and deployed outside of the original class environment.

- Develop a security mindset that anticipates adversarial exploits in software architecture and designs to expose potential vulnerabilities, mitigate design flaws, and ensure privacy and enhanced security of data and resources

User authentication, HTTPS/TLS, and removing MongoDB admin credentials from the code improved the security design of the application. Docker also improves the deployment structure because the services can communicate on an internal Docker network while only the needed frontend ports are exposed.

- Design, develop, and deliver professional-quality oral, written, and visual communications that are coherent, technically sound, and appropriately adapted to specific audiences and contexts.

The admin page and dashboard view are designed for different users. Animal shelters need a way to create, update, and maintain records, while rescue services need a dashboard view for reviewing and analyzing candidate animals. Maintaining both views shows the ability to present the same data in a way that fits the user and purpose.

While the Docker changes make the project more repeatable and easier to deploy, I found that it required a large amount of time to research the different images and how each service needed to be launched. Each service requires its own Dockerfile or configuration, and the Compose file must orchestrate how the frontend, backend, database, and dashboard services work together. This was one of the larger learning points for me because Docker makes deployment cleaner after it is working, but there is a lot of setup needed to get there.

The MongoDB source data also caused an issue. For the endpoint that creates a new animal record, I added a requirement that the animal ID should be unique. I found that MongoDB was not enforcing that requirement at first, and it took some time to research and diagnose the issue.

The source data already had duplicate animal IDs, so MongoDB could not enforce the unique index correctly. Deleting the duplicates from the seed file corrected the issue. This helped me better understand that database rules depend on the existing data already meeting the rule.

I have more experience with backend work and not as much with HTML and frontend design. Through research, it seemed that Vue.js would be simpler to implement than React. For a simple CRUD application, I found that Vue.js still had a learning curve, but the example code files were helpful for understanding the file structure and how the frontend should work. This enhancement helped me better understand how the frontend, backend API, database, and deployment pieces fit together in a full-stack application.